

Philosopher Debate Arena

Design and Qualitative Evaluation of a Prompt-Conditioned Multi-Agent Debate System

Sven Natterer & Simon Rönner
INF-3600 Generative Artificial Intelligence

May 2026

Abstract

This report presents the design and qualitative evaluation of *Philosopher Debate Arena*, an interactive multi-agent prototype in which large language models generate philosopher-inspired debates. The system allows users to select two philosopher personas, configure model backends and debate parameters, and generate a multi-round argumentative exchange on a selected topic. A separate judge component evaluates the generated debate according to a structured rubric, while optional summary, audio, and developer diagnostics support additional modes of interaction and inspection. The objective of the project is not to reconstruct historical philosophers with scholarly accuracy. Rather, the prototype investigates how prompt-based persona conditioning, configurable argumentation strategies, and LLM-based evaluation can be integrated into an interactive generative AI application. The evaluation indicates that the system can produce coherent and stylistically distinguishable debate interactions, but also reveals limitations concerning factual grounding, persona fidelity, judge reliability, and reproducibility.

1 Introduction

Large language models (LLMs) are capable of producing fluent argumentative text and can be conditioned through prompts to adopt particular roles, styles, and argumentative positions. These properties make them suitable for exploratory applications in which multiple model instances interact under different constraints. One possible application domain is philosophical debate, where contrasting positions, rhetorical style, and argumentative structure are central to the user experience.

This project investigates the use of prompt-conditioned LLM agents in an interactive debate environment. The implemented prototype, *Philosopher Debate Arena*, presents philosophical argumentation as a structured exchange between two philosopher-inspired agents. Each agent receives a persona description, a debate stance, an argumentation strategy, and a maximum response length. The resulting exchange is evaluated by one or more LLM-based judge agents.

The guiding research question is:

To what extent can prompt-conditioned LLM agents generate coherent, distinguishable, and interactive philosopher-inspired arguments in a configurable debate setting?

The project has both technical and educational relevance. From a technical perspective, it examines prompt engineering, multi-agent orchestration, model routing, UI state management, and automated evaluation. From an educational perspective, it explores whether abstract philosophical positions can be made more accessible through interactive debate. The prototype should therefore be understood as an experimental system for generative interaction design rather than as an authoritative philosophical simulation.

2 System Overview

The system was implemented as a Streamlit application with a staged interaction flow. Users first configure global settings, select or redraw a debate topic, choose two philosopher personas, define their strategies and model backends, and then generate a debate. After generation, the system displays the transcript, model attribution, judge result, optional written summary, optional browser-based speech output, and optional developer diagnostics.

The application separates the user interface from the debate engine. The UI manages session state, configuration, stage transitions, and rendering. The debate engine performs model calls, constructs prompts, records token usage when available, executes the debate loop, and runs judging and summarization. The current implementation supports Azure-compatible model deployments and local Ollama models.

2.1 Main Components

Component	Function
Streamlit frontend	Provides the staged user interface for settings, topic selection, character selection, debate display, summary, and diagnostics.
Philosopher personas	Define philosopher-inspired names, images, stances, and stylistic descriptions used for prompt conditioning.
Debate agents	Generate arguments for the assigned side of the topic according to persona, stance, strategy, history, and word limit.
Strategy module	Converts user-selected strategies into prompt instructions that influence argumentative form and tone.
Judge panel	Evaluates the transcript using one to three judge agents with configurable model backends and focus areas.
Summary module	Optionally generates a concise written summary of the debate and judgement.
Developer diagnostics	Optionally displays token usage metadata returned by the selected backend.
Audio output	Optionally exposes browser-based text-to-speech controls for replaying debate arguments.

Table 1: Functional overview of the prototype components.

3 Methodology

The project follows a prototype-oriented methodology. Instead of training a new model, it investigates how existing LLMs behave when embedded in a constrained interaction design. The implementation therefore focuses on prompt construction, role assignment, configurable model routing, and qualitative inspection of generated outputs.

3.1 Technical Challenges and API Integration

A significant technical challenge emerged during the integration of various LLM backends, particularly when using modern "reasoning" models such as DeepSeek-R1 or Phi-4. These models

often output internal "thinking traces" (e.g., within `<think>` or `<thought>` tags) before providing the final answer. While beneficial for the model's internal logic, these traces are not intended for the public debate arena and can significantly increase token consumption and API costs.

To address this, a robust post-processing pipeline was implemented in the debate engine. The cleaning function utilizes regular expressions to strip XML-style reasoning blocks, remove markdown artifacts, and filter out redundant headers (e.g., "Speaker:", "**Argument Opening:**") that models occasionally generate despite strict negative constraints in the system prompt. This ensures a clean user interface where only the philosopher's argument is visible.

A significant challenge in this context is the *unpredictability of model-specific formatting*. Despite explicit instructions and structural enforcement (such as requiring responses to be wrapped in `<public_response>` tags), different models tend to invent unique ways of labeling their output. Some models might repeat the debate topic as a title, others hallucinate conversational filler like "Here is my argument:", or they produce malformed tags with extra colons and whitespace. This variability makes it nearly impossible to anticipate every possible output format. Consequently, the cleaning logic had to evolve from simple keyword matching to an aggressive structural line-by-line filter that strips anything resembling a header or meta-instruction until the first substantive paragraph is reached. This highlights the persistent difficulty of achieving strictly deterministic formatting in non-deterministic LLM systems.

Furthermore, *UI formatting issues* emerged when integrating raw model text into custom HTML elements within Streamlit. Models naturally generate markdown paragraphs separated by double newlines (`\n\n`). When injecting this raw text directly into styled HTML paragraph tags (`<p>`), the Streamlit markdown engine interprets the double newlines as a signal to break out of the HTML container, rendering subsequent paragraphs with default unstyled formatting. This resulted in visual glitches where text alternated between custom CSS styling and default system fonts. The solution required explicitly processing all model outputs through a markdown-to-HTML converter before embedding them into structural `<div>` blocks.

Another major hurdle involved the *cognitive capacity of very small models* (e.g., the 1B parameter class). Enforcing complex, nested JSON schemas using tools like the `instructor` library proved too demanding for these models. The dual burden of adhering to strict JSON syntax while simultaneously generating creative philosophical content frequently led to "schema collapse," where models would regurgitate template placeholder strings from the prompt instead of generating novel arguments. To resolve this, strict JSON enforcement was abandoned for the core Philosopher agents in favor of natural raw text generation. Conversely, the Judge component—which strictly requires structured data (scores, numerical metrics, and categorical winners)—was restricted to more capable models (e.g., GPT-4.1 mini or Mistral Large) to ensure reliable JSON parsing.

Finally, *API timeouts* presented a significant obstacle, particularly with reasoning models and large local deployments. The default timeout threshold of 20–30 seconds was consistently exceeded when local environments (e.g., Ollama) loaded large models (such as the 14B parameter Phi-4) into limited hardware (e.g., 8GB VRAM) requiring slow CPU offloading. Similarly, API-based reasoning models often required 60–90 seconds to generate extensive internal `<think>` traces before returning the visible response. Expanding the client timeout windows to 180 seconds was necessary to ensure stability.

Another technical observation concerned the *refusal behavior* of smaller, highly-aligned models such as Phi-4 Mini. During testing, it was observed that these models would occasionally refuse to participate in the debate, providing a standard safety-refusal message. This behavior was triggered by a combination of the "debate" framing and specific persona descriptions, particularly Friedrich Nietzsche's "provocative" and "critical" stance. Smaller models, which are often

strictly tuned for politeness and conflict avoidance, may interpret the instruction to be "critical" in a "debate" as a request for hostile or harmful behavior, thus triggering an over-cautious safety refusal. To mitigate this, larger models or less restrictive system prompts are required to ensure consistent participation across all philosopher personas.

3.2 Model Benchmarking and Cost Optimization

To evaluate the system’s economic feasibility, various model classes were benchmarked. Testing focused on identifying low-cost or local alternatives that maintain sufficient debate quality without the high latency and cost of frontier models.

- **Local Models:** Testing with *Llama 3.2 (1B)* via Ollama provided a zero-cost baseline for rapid development and testing of UI logic.
- **Low-Cost Cloud Models:** *Phi-4 mini* and *GPT-4o mini* were identified as the most cost-efficient cloud backends, offering high speed and reliable instruction following at a fraction of the cost of larger models.
- **Reasoning Models:** *DeepSeek-V3* and *DeepSeek-R1* were tested for more complex philosophical argumentation. While they provide superior depth, their tendency to generate long internal traces required the aforementioned cleaning logic to prevent UI clutter and manage token budgets.

3.3 Agent Configuration

Each debate run contains two primary debate agents. For each agent, the system stores the selected philosopher persona, debate side, argumentation strategy, model provider, and model name. Before the debate starts, these values are copied into stable session-state keys to avoid losing widget-bound values during Streamlit stage transitions. This snapshot mechanism is important because debate generation occurs after the character selection widgets are no longer rendered.

The number of argument rounds is configurable in the start settings. One argument round corresponds to one generated contribution from each philosopher. For example, three rounds produce six debate turns before the judge panel evaluates the transcript. The maximum word count per philosopher contribution is also configurable, which allows the user to trade off response detail, latency, and token consumption.

3.4 Debate Generation

The debate engine receives the topic, number of rounds, selected strategies, and agent configurations. It iterates over the configured number of rounds and asks each agent to produce one response per round. Each prompt contains the debate topic, the agent’s assigned stance, the current transcript history, the selected strategy, and a word-limit instruction.

This design creates a controlled interaction structure: both agents respond to the same topic, each agent is repeatedly exposed to the evolving transcript, and the number of turns is externally constrained. The approach does not include long-term memory or retrieval-augmented grounding in the current version.

3.5 Evaluation Procedure

The prototype was evaluated qualitatively through functional testing of different configurations. The evaluation focused on whether the system produced coherent transcripts, whether persona-conditioned outputs were distinguishable, whether strategy selection influenced argument structure, and whether judge outputs were usable as feedback within the application.

Dimension	Variation	Evaluation Focus
Persona	Socrates, Nietzsche, Kant, Plato, Aristotle, Mill, Beauvoir	Stylistic recognizability and consistency.
Stance	For and against the selected topic	Ability to argue from an assigned position.
Strategy	Logical, emotional, rhetorical, or counterargument-oriented styles	Influence on structure, tone, and rebuttal behavior.
Rounds	One to five argument rounds	Effect on interaction depth and transcript length.
Model backend	Local and cloud-based model deployments	Coherence, latency, and reliability.
Judge panel	One to three judge agents	Stability and usefulness of automated evaluation.

Table 2: Qualitative evaluation dimensions considered during prototype testing.

4 Results

The implemented system generates structured multi-turn debate transcripts in which both agents address the same topic from opposing perspectives. In functional tests, the system maintained the selected debate roles across stage transitions and correctly propagated the selected philosopher personas into the loading and arena stages. This was a relevant implementation concern because Streamlit widget state can be lost when widgets are no longer displayed.

The generated arguments were generally coherent at the level of local discourse. Agents typically responded to the topic, followed the assigned side, and incorporated parts of the preceding transcript. The selected strategy also influenced the surface structure of the output. Logical strategies tended to produce more explicit premise-conclusion structures, whereas rhetorical or emotional strategies produced more expressive language.

Persona conditioning produced recognizable stylistic differences in many cases. For instance, some agents generated abstract and principle-oriented arguments, while others used more provocative or existential language. However, these differences should be interpreted cautiously. The system does not verify generated claims against primary philosophical texts, and the resulting personas may reflect simplified cultural associations rather than accurate philosophical positions.

The judge component provided useful game-oriented feedback. It produced a winner, reasoning, and metric-based scores. Multi-judge evaluation made the result more transparent by showing distinct focus areas before aggregating scores. Nevertheless, the judge output remained model-generated and should not be interpreted as an objective assessment of philosophical quality.

5 Discussion

The prototype demonstrates that prompt-conditioned LLM agents can be integrated into a coherent interactive debate workflow. The staged UI makes the interaction understandable, while configurable settings allow users to control summary generation, response length, number of argument rounds, audio output, and developer diagnostics. These controls are important because model-based debate generation involves trade-offs among output richness, latency, token cost, and usability.

The system is most convincing as an exploratory educational interface. It allows users to compare how different philosopher-inspired personas and strategies shape argumentative responses. The interaction design therefore supports engagement with philosophical themes, even when the generated content remains approximate and non-authoritative.

At the same time, the system highlights common limitations of LLM-based persona simulation. Prompt conditioning can steer style and perspective, but it does not guarantee factual accuracy, historical fidelity, or stable reasoning. The lack of retrieval-augmented grounding means that the system relies primarily on model-internal knowledge and the short persona descriptions provided in the prompt context. Consequently, the output may overrepresent familiar stereotypes or produce plausible but imprecise philosophical claims.

The judge module raises a second methodological issue. Because the evaluator is also an LLM, it may reward fluency, confidence, and rhetorical clarity more strongly than philosophical rigor. In the current prototype, the judge should therefore be considered part of the interaction design rather than a valid scientific measurement instrument.

6 Limitations

Several limitations constrain the interpretation of the prototype.

First, the philosopher personas are generated through prompt conditioning rather than through retrieval from primary or secondary philosophical sources. This limits authenticity and makes the output sensitive to model pretraining biases. A philosopher-inspired agent may therefore reproduce popular associations rather than a nuanced reconstruction of the philosopher’s work.

Second, the evaluation is qualitative and exploratory. The current report does not present a controlled user study, statistical comparison, or benchmark-based evaluation. As a result, claims about coherence, recognizability, and educational value should be understood as observations from prototype testing rather than as generalizable empirical findings.

Third, the system is sensitive to prompt wording and backend choice. Different models may produce substantially different levels of coherence, style consistency, and judgement quality. Reproducibility therefore requires careful documentation of selected models, prompts, settings, and debate topics.

Fourth, the browser-based audio output improves interaction but does not provide high-quality character voices. It depends on the voices available in the user’s browser or operating system and should be considered an accessibility and presentation feature rather than a realistic voice simulation.

7 Future Work

Future development should focus on grounding, evaluation, and richer interaction.

A retrieval-augmented generation component could improve philosophical authenticity by retrieving relevant passages from curated primary or secondary texts before each agent response. This would allow agents to refer to source material rather than relying solely on model-internal knowledge. Such a feature would require careful source selection, chunking, retrieval, citation handling, and prompt integration.

The evaluation methodology could be strengthened through human ratings. Participants could assess debate transcripts according to coherence, persona consistency, persuasiveness, educational value, and perceived authenticity. These ratings could then be compared with the judge agent’s scores to investigate whether the automated evaluation aligns with human judgement.

The multi-agent structure could also be extended. Instead of one model call per philosopher turn, each side could contain an internal team of agents, such as a strategist, critic, and final speaker. This would increase computational cost but could improve argument quality and create more complex interactions.

Finally, the audio system could be upgraded from browser-based speech synthesis to local text-to-speech generation. Local models such as Kokoro or Piper could provide more consistent voices without requiring an external API, while expressive models or curated sound effects could support more theatrical presentation.

8 Conclusion

Philosopher Debate Arena demonstrates how prompt-conditioned LLM agents can be combined with an interactive UI, configurable debate settings, automated judging, and optional diagnostics to create a multi-agent debate prototype. The system can generate coherent and stylistically varied philosopher-inspired exchanges, especially when persona, stance, strategy, and debate length are clearly constrained.

The prototype should not be interpreted as a historically accurate simulation of philosophers or as an objective evaluator of philosophical reasoning. Its primary contribution lies in demonstrating an interactive design pattern for generative AI systems: role-conditioned agents can be orchestrated into structured debates, inspected through a judge component, and exposed to users through adjustable settings. The project therefore provides a foundation for future work on grounded philosophical agents, richer multi-agent collaboration, and more rigorous evaluation of LLM-generated argumentative interaction.